



Participante: Luis Ramon Reyes Volquez

Técnico: Ciberseguridad

Módulo: Shell Scripting

Facilitador: Kevin Feliz Hernández

Centro: Cyberclass

Práctica: Investigación de Shell Scripting

Descripción: Esta es una investigación de conceptos avanzados de Shell Scripting

Github: <https://github.com/elchecho3000-blip/Ciberseguridad-bootcamp/tree/main/Shell%20Scripting>

Introducción

El propósito de este documento es analizar cuatro áreas clave de la administración de sistemas y scripting en Windows:

1. Políticas de ejecución en PowerShell.
2. Uso de WSL2 para ejecutar Bash.
3. Diferencias entre tuberías en CMD y PowerShell.
4. Automatización en la nube con Azure Cloud Shell.

Políticas de Ejecución y Seguridad en PowerShell

- **Execution Policy:** mecanismo de seguridad que determina si los scripts pueden ejecutarse y bajo qué condiciones.
- **Modos principales:**
 - **Restricted:** bloquea todos los scripts, solo comandos interactivos.
 - **RemoteSigned:** permite scripts locales sin firma, pero exige firma digital para los descargados de internet.
 - **Unrestricted:** permite todos los scripts, mostrando advertencias en algunos casos.
- **Razón de Microsoft:** evitar la ejecución accidental de scripts maliciosos y establecer reglas básicas de seguridad.
- **Riesgo:** ejecutar scripts descargados sin revisión puede abrir puertas a malware, robo de credenciales o alteración del sistema.

Windows Subsystem for Linux (WSL2)

- **WSL2:** subsistema que integra un kernel real de Linux en Windows mediante una VM ligera, con mejor rendimiento que WSL1.
- **Permite:** instalar distribuciones como Ubuntu y ejecutar scripts de Bash directamente en Windows.
- **Escenarios donde Bash es preferible:**
 - Administración de servidores Linux.
 - Uso de herramientas clásicas (grep, sed, awk).
 - Automatización en entornos DevOps y despliegues web.

El uso de "Pipe" y Objetos

- **CMD**: tuberías transmiten texto plano (bytes), lo que obliga a usar herramientas externas para filtrar.
- **PowerShell**: tuberías transmiten **objetos .NET**, con propiedades y métodos, lo que permite manipulación más robusta.
- **Ejemplo práctico**: filtrar servicios detenidos.

```
Get-Service | Where-Object { $_.Status -eq 'Stopped' }
```

Este comando devuelve solo los servicios cuyo estado es “Stopped”.

Automatización en la Nube (Azure Cloud Shell)

- **Azure Cloud Shell:** entorno interactivo accesible desde navegador, con autenticación automática y soporte para **Bash o PowerShell**.
- **Uso de PowerShell en la nube:** los mismos cmdlets aprendidos en local (ej. Get-AzVM, Set-AzContext) se aplican para administrar máquinas virtuales, redes y bases de datos en Azure.
- **Ventaja:** no requiere instalación ni configuración local; siempre actualizado y seguro.

Aporte Personal

De todas estas tecnologías, considero que **WSL2** es la más útil para mi carrera profesional. Permite combinar la potencia de Linux con la comodidad de Windows, facilitando el aprendizaje de Bash y la interoperabilidad en entornos mixtos, algo esencial en administración de sistemas y ciberseguridad.

Bibliografía

- Microsoft Learn – about_Execution_Policies ([learn.microsoft.com in Bing](#))
- Microsoft Learn – Set-ExecutionPolicy ([learn.microsoft.com in Bing](#))
- PcHardwarePro – *Bash Scripting on Windows with WSL*
- Windows Noticias – *Cómo crear scripts de Bash en Windows con WSL*
- LinuxSimply – *Bash vs PowerShell Quick Comparison*
- Microsoft Learn – Where-Object ([learn.microsoft.com in Bing](#))
- Microsoft Docs – *Azure Cloud Shell Overview*